

CS 639 Spring 2026 HW 5: Alignment

Due: May 6, 2026 11:59 pm

In this assignment, you will build a preference-based alignment pipeline for a small decoder-only transformer. The task is a controlled knowledge-seeking QA problem: when the context contains evidence, the model should answer correctly; when the evidence is missing, the aligned model should abstain with "I don't know."

You will implement the key objectives behind supervised fine-tuning (SFT), reward modeling with pairwise preferences, and direct preference optimization (DPO). In this assignment, you will study reward-model reranking and DPO in a controlled finite-candidate alignment setting.

Instructions.

- The starter notebook is designed to run on Google Colab with a T4 GPU.
- Fill in only the parts marked TODO.
- Do not modify the provided training loops or helper functions unless explicitly instructed.

What the notebook already provides.

- Synthetic data generation for supported and unsupported prompts.
- A word-level tokenizer and batching utilities.
- A small decoder-only transformer policy and a reward model.
- Training loops, evaluation code, and behavior metrics.
- Local sanity-check cells after each required implementation task.

Success criteria.

- **Part A / Part B:** SFT loss should decrease; reward-model loss should decrease; reward-model validation preference accuracy should rise substantially above chance; the reward-model validation summary score should also rise.
- **Part B:** increasing unsupported preference data should make abstention more visible; increasing preference noise should weaken reward-model quality on at least one behavior metric.
- **Part C:** you should evaluate DPO honestly. A lower DPO loss does not by itself guarantee that the final finite-candidate behavior has changed.

Validation summary score. The notebook tracks

$$\text{summary_score} = \frac{1}{2} (\text{supported_acc} + \text{unsupported_abstain_rate}).$$

This score is used only as a monitoring metric. Final reporting should always use the separate metrics.

Model and task configuration. We use a word-level tokenizer and a small decoder-only transformer. The synthetic world uses two attributes (color and habitat), and the model must answer only when the requested fact is supported by the context.

Quantity	Default value
Embedding dimension	128
Number of heads	4
Number of transformer layers	4
Block size	192
SFT training iterations	1000
Reward-model training iterations	300
DPO training iterations	700
Default unsupported fraction (RM)	0.60
Supported <code>correct > abstain</code> copies	3
Supported wrong-answer negatives	all 3
Unsupported hallucinated negatives	all 4
Default DPO β	0.80

Submission. Submit a ZIP file containing:

- **Notebook (.ipynb):** your completed notebook with all outputs visible.
- **Writeup (.pdf):** your written answers, plots, tables, and analysis for every question marked [Writeup] or [Both].

Each subsection below is labeled with its deliverable:

- [Code] — fill in the notebook code.
- [Writeup] — include the requested plots, tables, and written analysis in the PDF.
- [Both] — run the notebook code and include the relevant outputs and analysis in the PDF.

Part A: Core Alignment Implementation

In this part, you will implement the key scoring and objective functions behind response-only SFT, sequence scoring under a policy, reward modeling from pairwise preferences, and the validation metrics used to monitor behavior.

A1: Response-Only LM Loss (10 points) [Code]

Implement `masked_cross_entropy(logits, labels)`.

- Ignore prompt positions marked with label `-100`.
- Return a scalar mean loss.

A2: Sequence Log-Probabilities (10 points) [Code]

Implement `sequence_logprob(logits, labels)`.

- Sum token log-probabilities over response positions only.
- Return one scalar sequence log-probability per example.

A3: Last-Token Pooling (10 points) [Code]

Implement `last_token_pool(hidden_states, attention_mask)`.

- Use the final non-padding token indicated by the attention mask.

A4: Reward Model Forward Pass (10 points) [Code]

Implement `RewardModel.forward`.

- The reward model should score an entire (x, y) sequence with a scalar.

A5: Bradley–Terry Preference Loss (10 points) [Code]

Implement the pairwise reward-model objective

$$-\log \sigma(r(x, y_{\text{chosen}}) - r(x, y_{\text{rejected}})) .$$

Return a scalar mean loss.

A6: Preference Accuracy (10 points) [Code]

Implement `preference_accuracy(chosen_scores, rejected_scores)`.

- This should measure the fraction of pairs where the chosen response receives the higher score.

A7: Training and Validation (10 points) [Both]

Train the SFT policy and the reward model using the provided notebook cells.

In your writeup, include:

- SFT train/validation loss curves,
- reward-model train/validation loss curves,
- reward-model validation preference-accuracy curve,
- reward-model validation summary-score curve,
- a final metric table with supported-question accuracy, unsupported-question abstain rate, and unsupported-question hallucination rate.

Part A sanity check. Before moving on:

- SFT loss should decrease.
- Reward-model loss should decrease.
- Reward-model validation preference accuracy should rise substantially above chance.
- Reward-model validation summary score should also rise.

Part B: Alignment Experiments

Run the provided ablations and interpret the effect of preference-data design on aligned behavior.

B1: Varying the Amount of Unsupported Preference Data (5 points) [Writeup]

Keep the full-candidate pair construction fixed, and vary the fraction of unsupported preference prompts kept in the training set:

- 0% of unsupported preference prompts,
- 2.5% of unsupported preference prompts,
- 5% of unsupported preference prompts.
- 10% of unsupported preference prompts.
- 20% of unsupported preference prompts.

In your writeup:

- include the unsupported-question abstain-rate plot,
- report the final metrics for each setting,
- explain what this says about learning abstention from preference supervision when abstention is compared against *all* factual candidates.

B2: Varying Preference Noise (5 points) [Writeup]

Keep the default pair construction fixed, and train three reward models with different amounts of preference-label noise:

- 0% flipped pairs,
- 5% flipped pairs,
- 10% flipped pairs.

In your writeup:

- include the combined metric plot,

- report the final metrics for each setting,
- explain how preference quality changes the supported-vs-unsupported tradeoff.

Part B sanity check.

- In B1, adding unsupported preference data should make abstention more visible.
- In B2, increasing preference noise should usually weaken reward-model quality on at least one behavior metric.

Part C: Direct Preference Optimization — Comparison and Failure Analysis

In this part, you will implement DPO as a policy-level alternative to reward-model reranking. However, unlike Part A and Part B, you should *not* assume that DPO will necessarily match reward-model reranking in this controlled finite-candidate setting. One goal of this part is to compare the methods honestly and analyze any mismatch between a decreasing DPO objective and the final behavior metrics.

C1: SFT vs. Reward-Model Reranking vs. DPO (5 points) [Writeup] (Answer this question after completing C2)

Fill in a comparison table for:

- supported-question accuracy,
- unsupported-question abstain rate,
- unsupported-question hallucination rate.

Then answer:

- Which method works best in this controlled finite-candidate setting?
- If DPO underperforms reward-model reranking, why might that happen in this setup?
- Why can a policy objective improve while the final finite-candidate behavior remain unchanged?

C2: DPO Loss and Training (25 points) [Both]

Implement `dpo_loss` using policy and reference sequence log-probabilities. Recall the DPO objective:

$$-\log \sigma \left(\beta \left[(\log \pi_{\theta}(y_c | x) - \log \pi_{\theta}(y_r | x)) - (\log \pi_{\text{ref}}(y_c | x) - \log \pi_{\text{ref}}(y_r | x)) \right] \right).$$

The notebook reports the **final checkpoint** and plots the DPO loss curve. Final evaluation should use the reported supported-question accuracy, unsupported-question abstain rate, and unsupported-question hallucination rate.

In your writeup:

- include the DPO loss curve,
- report the final DPO behavior metrics,
- explain whether the final finite-candidate behavior changed,
- explain what it means if the DPO loss decreases but the final behavior metrics do not change.

Part C sanity check. A decreasing DPO loss is *not* by itself the target outcome. The key question is whether the final candidate behavior changed.

Final reminder. You are not allowed to change the pre-provided training loops or helper functions except where the notebook explicitly asks you to implement a TODO. Focus on the requested functions, use the local sanity checks to debug early, and make sure your notebook outputs are complete and readable before submission.